1. **Aim of the practical:** Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.
2. Query an alphabetically ordered list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S). 2. Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in ascending order

2. Objective:

Proficiency Development: Enhance skills in using Common Table Expressions (CTEs) and window functions for effective data analysis.

Date Manipulation: Gain expertise in manipulating dates and using aggregation techniques to analyze project timelines and durations.

Insight Extraction: Learn to extract meaningful insights from task-related data to support project management decisions.

3. Commands/Steps:
A)

Step-1: Created Database:
```
mysql> CREATE DATABASE Task;
Query OK, 1 row affected (0.04 sec)
```

Step-2: Use this database:
```
mysql> use Task;
Database changed
```

Setp-3: Created Task_Table:
```
mysqlmysql> CREATE TABLE Task_Table (
        Task_ID INT,
        Start_Date DATE,
        End_Date DATE
Query OK, 0 rows affected (0.10 sec)
```

Step-4: Inserted 7 records into the Task_Table with 3 Attributes .
```
mysql> INSERT INTO Task_Table (Task_ID, Start_Date, End_Date)
    VALUES
    (1, '2015-10-01', '2015-10-02'),
    (2, '2015-10-02', '2015-10-03'),
    (3, '2015-10-03', '2015-10-04'),
    (4, '2015-10-13', '2015-10-14'),
    (5, '2015-10-14', '2015-10-15'),
    (6, '2015-10-28', '2015-10-29'),
    (7, '2015-10-30', '2015-10-31');
Query OK, 7 rows affected (0.02 sec)
Records: 7  Duplicates: 0  Warnings: 0
```

Steps-5: Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

```
WITH Project_Groups AS (
   SELECT
      Task_ID,
      Start_Date,
      End_Date,
      CASE
         WHEN Start_Date = DATE(LAG(End_Date) OVER (ORDER BY Start_Date), '+1
day') THEN 0
         ELSE 1
      END AS New_Project_Flag
   FROM Projects
),
Project_Numbered AS (
   SELECT
      Task_ID,
      Start_Date,
      End_Date,
      SUM(New_Project_Flag) OVER (ORDER BY Start_Date) AS Project_ID
   FROM Project_Groups
)
SELECT
   MIN(Start_Date) AS Project_Start_Date,
   MAX(End_Date) AS Project_End_Date
FROM Project_Numbered
GROUP BY Project_ID
ORDER BY
```

```
   JULIANDAY(MAX(End_Date)) - JULIANDAY(MIN(Start_Date)),  -- Order by project duration
   MIN(Start_Date);
```

B)

Step-1: Created Database:

```
create database occup;
Query OK, 1 row affected (0.01 sec)
```

Step-2: Use this database:

```
mysql> use occup;
Database changed
```

Setp-3: Created Occupatons table:

Database changed

```
mysql> CREATE TABLE Occupations (
        Name VARCHAR(255),
        Occupation VARCHAR(255)
    );
Query OK, 0 rows affected (0.03 sec)
```

Step-4: Inserted 9 records into the Task_Table with 2 Attributes .

```
mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Samantha', 'Doctor');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Julia', 'Actor');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Maria', 'Actor');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Meera', 'Singer');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Ashley', 'Professor');
```

```
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Christeen', 'Professor');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Jane', 'Actor');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Jenny', 'Doctor');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Occupations (Name, Occupation) VALUES ('Priya', 'Singer');
Query OK, 1 row affected (0.00 sec)
```

Steps-5: Query an alphabetically ordered list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S). 2. Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in ascending order,

List of Names and Occupations

```
mysql> SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')') AS formatted_output
   -> FROM Occupations
   -> ORDER BY Name ASC;
```

Count of Each Occupation

```
mysql> SELECT
   ->     CONCAT('There are a total of ', COUNT(*), ' ', LOWER(Occupation), 's.') AS
formatted_output
   -> FROM Occupations
   -> GROUP BY Occupation
   -> ORDER BY COUNT(*) ASC, LOWER(Occupation) ASC;
```

5. Result:
A)

Task_Table

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

Final Output

Output

Project_Start_Date	Project_End_Date
2015-10-01	2015-10-02
2015-10-13	2015-10-14
2015-10-14	2015-10-15
2015-10-28	2015-10-31

B)

Occupations Table

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashley	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

List of Names and Occupations

formatted_output
Ashley(P)
Christeen(P)
Jane(A)
Jenny(D)
Julia(A)
Maria(A)
Meera(S)
Priya(S)
Samantha(D)

Count of Each Occupation

formatted_output
There are a total of 2 doctors.
There are a total of 2 professors.
There are a total of 2 singers.
There are a total of 3 actors.

6. Learning outcomes (What I have learned):

a) Learn how to manipulate and compare date values in SQL

b) Understand how to extract and analyze project-related data, enabling insights into project durations and timelines.

c) Learned data aggregation using GROUP BY clauses to summarize information based on specific criteria,

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet		8 Marks
2.	Viva		10 Marks
3.	Simulation		12 Marks
	Total		30 Marks

Teacher Signature